

Vulnerability Assessment Report

Week 1 — User Management System
(Express + EJS + SQLite)

Author: Maryam Mueen
Intern ID: DHC-1151

1 Scope

The target is `vulnerable-app/`, a small User Management System running on `http://localhost:3000`. Endpoints assessed: `/`, `/signup`, `/login`, `/profile`, `/users`, `/logout`. Authentication uses a plain integer cookie (`uid`). Storage is in-memory SQLite, seeded with `admin / admin123`.

Tools used. Manual testing via browser developer tools, `curl`, and OWASP ZAP automated active scan.

2 Summary of Findings

The assessment surfaced nine findings: two Critical, two High, four Medium, and one Low.

#	Vulnerability	OWASP Top 10 (2021)	Severity
V-01	SQL Injection on login	A03 — Injection	Critical
V-02	Stored XSS in profile bio	A03 — Injection	High
V-03	Plaintext password storage	A02 — Crypto Failures	Critical
V-04	Missing security HTTP headers	A05 — Misconfiguration	Medium
V-05	Server framework leak <code>X-Powered-By</code>	A05 — Misconfiguration	Low
V-06	Session cookie not flagged <code>HttpOnly/Secure/SameSite</code>	A07 — Auth Failures	High
V-07	Public <code>/users</code> JSON leaks emails	A01 — Broken Access Control	Medium
V-08	No input validation	A04 — Insecure Design	Medium
V-09	No rate limiting on auth endpoints	A07 — Auth Failures	Medium

3 Detailed Findings

3.1 V-01 — SQL Injection on `/login`

Severity: Critical | **OWASP:** A03 — Injection

Location

The login handler in `vulnerable-app/server.js` builds the query by string concatenation:

```
const sql =
  "SELECT * FROM users WHERE username = '" +
  username + "' AND password = '" +
  password + "'";
```

Reproduction

```
curl -i -X POST http://localhost:3000/login \
  --data-urlencode "username=admin' OR '1'='1" \
  --data-urlencode "password=anything"
```

Observed response:

```
HTTP/1.1 302 Found
Set-Cookie: uid=1; Path=/
Location: /profile
```

Impact

Complete authentication bypass. The attacker logs in as `admin` (user id 1) without knowing the password.

Recommendation

Use parameterised queries (`?` placeholders). `better-sqlite3` supports them natively.

3.2 V-02 — Stored XSS in profile bio

Severity: High | **OWASP:** A03 — Injection

Location

`vulnerable-app/views/profile.ejs` renders the bio with the unescaped `<%- %>` tag:

```
<div class="bio-box"><%- user.bio %></div>
```

Reproduction

Log in, set the bio to `<script>alert('XSS');</script>`, save, then revisit `/profile`. The alert fires on every page-load and on any other user who views this profile.

Impact

Session theft (the cookie has no `HttpOnly` flag, so the attacker's `<script>` can read `document.cookie` and exfiltrate it), defacement, and phishing.

Recommendation

Switch every `<%- %>` to `<%= %>` (HTML-escaped output), and add a Content-Security-Policy header that blocks inline scripts.

3.3 V-03 — Plaintext password storage

Severity: Critical | **OWASP:** A02 — Cryptographic Failures

Location

The signup handler `INSERTs` the password directly from the request body, untransformed.

Reproduction

Anyone with read access to the SQLite file — or anyone who reads `/users` after a small tweak — sees raw passwords. Confirmed by inspecting the seeded `admin` row: `password = 'admin123'`.

Impact

A database leak immediately exposes every user's credentials. Most users reuse passwords across sites.

Recommendation

Hash with `bcrypt.hash(password, 12)` on signup; verify with `bcrypt.compare(...)` on login.

3.4 V-04 — Missing security HTTP headers

Severity: Medium | **OWASP:** A05 — Security Misconfiguration

Reproduction

```
$ curl -I http://localhost:3000/  
HTTP/1.1 200 OK
```

```
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
...
```

No Content-Security-Policy, no Strict-Transport-Security, no X-Frame-Options, no X-Content-Type-Options, no Referrer-Policy.

Impact

The site can be iframed for click-jacking, MIME-sniffed, and inline scripts run freely.

Recommendation

`app.use(helmet())` with a strict CSP.

3.5 V-05 — X-Powered-By: Express leak

Severity: Low | **OWASP:** A05 — Security Misconfiguration

The X-Powered-By header tells an attacker exactly which framework family is in play, narrowing CVE research.

Recommendation

`app.disable('x-powered-by')` (helmet also does this).

3.6 V-06 — Insecure auth cookie

Severity: High | **OWASP:** A07 — Identification & Authentication Failures

Location

The login handler does `res.cookie('uid', user.id)`. The cookie is:

- not signed: anyone can edit it to impersonate any user (try `document.cookie = "uid=1"` to become admin),
- not HttpOnly: readable by JS, so XSS can steal it,
- not Secure: sent over plain HTTP,
- no SameSite: vulnerable to CSRF.

Recommendation

Issue a signed JWT cookie with `httpOnly: true, sameSite: 'strict', secure: true` in production.

3.7 V-07 — Public /users endpoint leaks emails

Severity: Medium | **OWASP:** A01 — Broken Access Control

GET `/users` returns the entire users table (id, username, email, bio) to unauthenticated callers. Reproduced:

```
$ curl http://localhost:3000/users
[{"id":1,"username":"admin","email":"admin@example.com",...}]
```

Recommendation

Require authentication, project only the columns the page needs (no email), and paginate.

3.8 V-08 — No input validation

Severity: Medium | **OWASP:** A04 — Insecure Design

Signup accepts empty usernames, `not-an-email`, single-character passwords, and 10 KB of HTML in `bio`.

Recommendation

Validate with the `validator` library (`isEmail`, `isAlphanumeric`, `isStrongPassword`, `length caps`).

3.9 V-09 — No rate limiting

Severity: Medium | **OWASP:** A07 — Identification & Authentication Failures

`/login` and `/signup` can be hit unboundedly — trivial credential stuffing and brute force.

Recommendation

Apply `express-rate-limit` (20 requests per 15 minutes per IP) on `/login` and `/signup`.

4 Areas of Improvement

1. Move to parameterised SQL across the entire codebase.
2. Hash passwords with `bcrypt` (`cost` ≥ 12).
3. Replace the plaintext cookie with a JWT, flagged `HttpOnly` + `SameSite=Strict` + `Secure`.
4. Apply `helmet()` and disable `X-Powered-By`.
5. Validate every input at the route boundary.
6. Escape every variable in templates (`<%= %>`).
7. Authenticate `/users`, project minimal columns.
8. Add rate limiting to all auth endpoints.
9. Add structured logging (`winston`) for auth events, errors, and suspicious activity.
10. Set `NODE_ENV=production` and stop returning stack traces on errors.

Next. All of the above are implemented in `secure-app/`. See Report 02 (Security Fixes Applied) for the diff-by-diff walk-through and Report 05 (Final Report) for the wrap-up.